Lab 8: OpenTelemetry

Observability for the AI-based Village Pond Planning System

Rishi Kharya

October 16, 2026

Application (sys4): http://10.1.75.79:5228/
Observability backend – OpenObserve (sys2): http://10.1.75.79:5226/
Source code: https://github.com/krish1809/village-pond (backend/telemetry.py)

1 What I set out to do

For this lab I added OpenTelemetry to my Assignment 1 project, the Village Pond Planning System. It is a FastAPI service that takes an area selected on a map, downloads an elevation model for it from OpenTopography, routes water across the terrain, ranks candidate pond sites, looks up rainfall from Open-Meteo and works out how much water each pond could collect. That is a good fit for observability: a single request crosses two external services and around ten processing stages, and when it is slow or fails, the plain request log does not say why.

The goal was to capture all three kinds of telemetry — **traces**, **metrics** and **logs** — from the real running application on the lab server, send them to a proper backend, and then actually use what was collected to explain how the system behaves.

2 How the implementation is organised

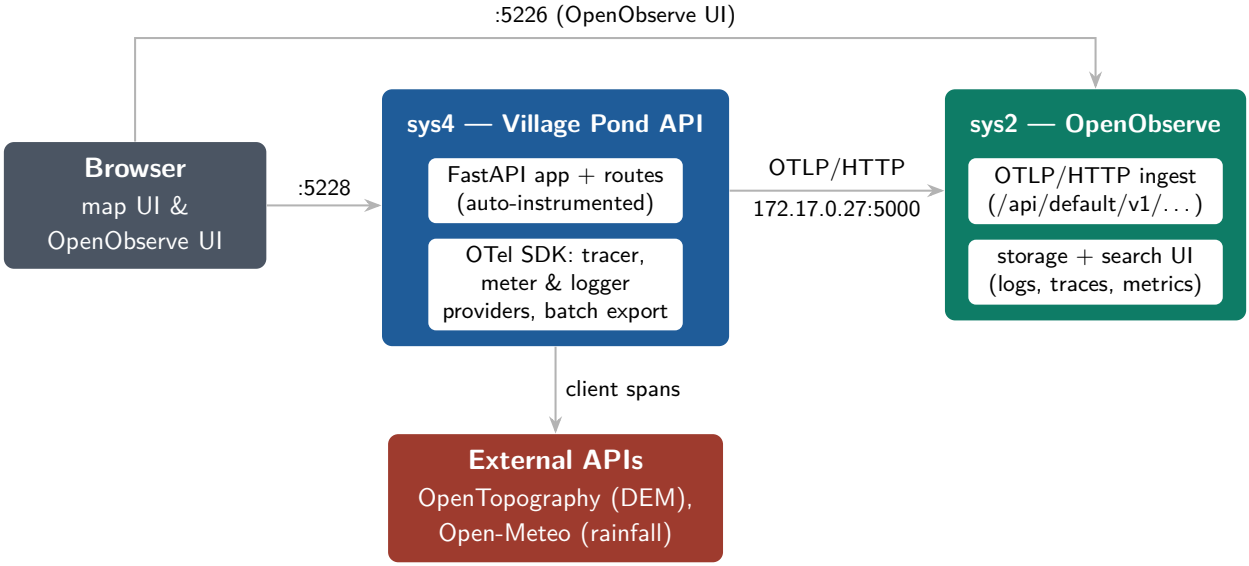


Figure 1: Where the telemetry comes from and where it goes. The app on sys4 exports over the internal lab network to OpenObserve on sys2; I view the data in the browser through sys2's forwarded port.

I kept the observability code separate from the analysis code. A single new module, `backend/telemetry.py`, installs the OpenTelemetry SDK once when the app starts. The rest of the code only talks to the OpenTelemetry *API* (`opentelemetry.trace` and `opentelemetry.metrics`), which does nothing until the SDK is installed. This is the pattern OpenTelemetry recommends for instrumenting existing code, and it had a practical benefit: my 71 existing tests never install the SDK, so they ran unchanged.

Everything is exported as **OTLP over HTTP** and configured only through the standard environment variables (`OTEL_EXPORTER_OTLP_ENDPOINT`, `OTEL_EXPORTER_OTLP_HEADERS`, `OTEL_SERVICE_NAME`). The code has no idea which backend is on the other end, so OpenObserve could be swapped for Jaeger, Grafana Tempo or an OpenTelemetry Collector without touching the application. If no endpoint is set, telemetry simply stays off.

Deployment on the lab systems

The lab gave us four small containers, each capped at 512 MB of RAM, with one port per container forwarded to the outside. The app already ran on sys4. I put the observability backend on **sys2**, the container with the most free memory and an unused forwarded port, so the two would not compete for sys4's 512 MB. The containers share an internal Docker network, so sys4 exports straight to `172.17.0.27:5000`, while I open the OpenObserve UI from my laptop through `http://10.1.75.79:5226/`. OpenObserve is a single binary that ingests and displays all three signals behind one port, which is exactly what a one-port, low-memory container needs. I started it with small worker and cache limits (the container reports 120 CPUs, and the defaults size themselves from that). It started at about 200 MB and, with the test traffic flowing in, sys2's container stayed at roughly 290 MB of real (anonymous) memory out of its 512 MB.

3 Tools, libraries and backend

Component	Role
`opentelemetry-api` / `-sdk` 1.45.1	Tracer, meter and logger providers; batch span and log processors; periodic metric reader
`opentelemetry-exporter-otlp-proto-http` 1.45.1	Sends spans, metrics and log records as OTLP/protobuf over HTTP
`opentelemetry-instrumentation-fastapi` 0.66b1	Automatic server span and HTTP metrics for every request
OpenObserve v1.1.0 (on sys2)	Backend that stores, searches and visualises logs, traces and metrics; supports SQL and PromQL queries
FastAPI + uvicorn on Python 3.14 (sys4)	The instrumented Village Pond API

Table 1: Libraries and services used.

4 What each signal captures

Traces. The FastAPI instrumentation creates one server span per request (method, route, status code, duration). Inside it I open a child span for every stage of the pipeline using a small helper, `telemetry.stage()`: the elevation download, contour generation, flow routing, drainage extraction, site ranking, the rainfall lookup, one span per candidate pond, and the PNG render when an image is requested. Each span carries attributes that matter for this domain, such as the grid size, whether the elevation model came from the cache, the rainfall source, and each candidate's catchment area, pond area and collectable volume. The two calls to external services get their own

client spans. I created these by hand instead of auto-instrumenting `httpx`, because the OpenTopography URL carries my API key in its query string; the spans record only the host, path, DEM type and response status, so the key never reaches the backend. Every response also returns its trace ID in an `X-Trace-Id` header, and the web page shows it under the results, so a slow request seen in the browser can be looked up directly.

Metrics. Metrics are pushed every 10 seconds. The FastAPI instrumentation provides HTTP request duration, request and response size and active requests. I added the application-specific instruments in Table 2. The memory gauges were not an afterthought: with a 512 MB hard limit, memory is the resource most likely to take this service down.

Instrument	Type	What it tells me
`pond.analyses`	counter	Analyses per endpoint and outcome (ok, invalid_request, upstream_error, error)
`pond.pipeline.stage.duration`	histogram	Time spent in each pipeline stage
`pond.dem.requests`	counter	Elevation requests served from the cache vs. fetched
`pond.upstream.errors`	counter	Failures of external services, by service and kind (e.g. rate limit)
`pond.rainfall.lookups`	counter	Rainfall from the live archive vs. the offline fallback
`pond.candidates.found`	histogram	Pond sites returned per analysis
`pond.collectable_volume`	histogram	Collectable water volume of suggested ponds, by rank
`process.memory.rss`	gauge	Resident memory of the API process
`container.memory`	gauge	Container (cgroup) memory used vs. its 512 MB limit

Table 2: Custom metrics added to the application.

Logs. The app's Python `logging` output is bridged to OpenTelemetry with a `LoggingHandler`. I log the events that matter — request received, DEM ready, analysis complete with the headline result, warnings when a request is invalid or rainfall falls back to the default, and errors from external services — with structured attributes (bounding box, durations, outcome). Because each record is written inside a span, it carries that span's `trace_id` and `span_id` automatically, which is what links a log line to the full trace of the request that produced it. I deliberately kept the `httpx` library logger quiet, since it logs every request URL — including the one with the API key.

5 Results from the running system

All screenshots below are from the deployed application on sys4, exporting to OpenObserve on sys2. The traffic was a mix of real analyses: contour-file uploads, map-area analyses (one fresh download and several cached repeats), PNG renders and deliberately invalid requests, plus one analysis run from the browser at the end.

Traces

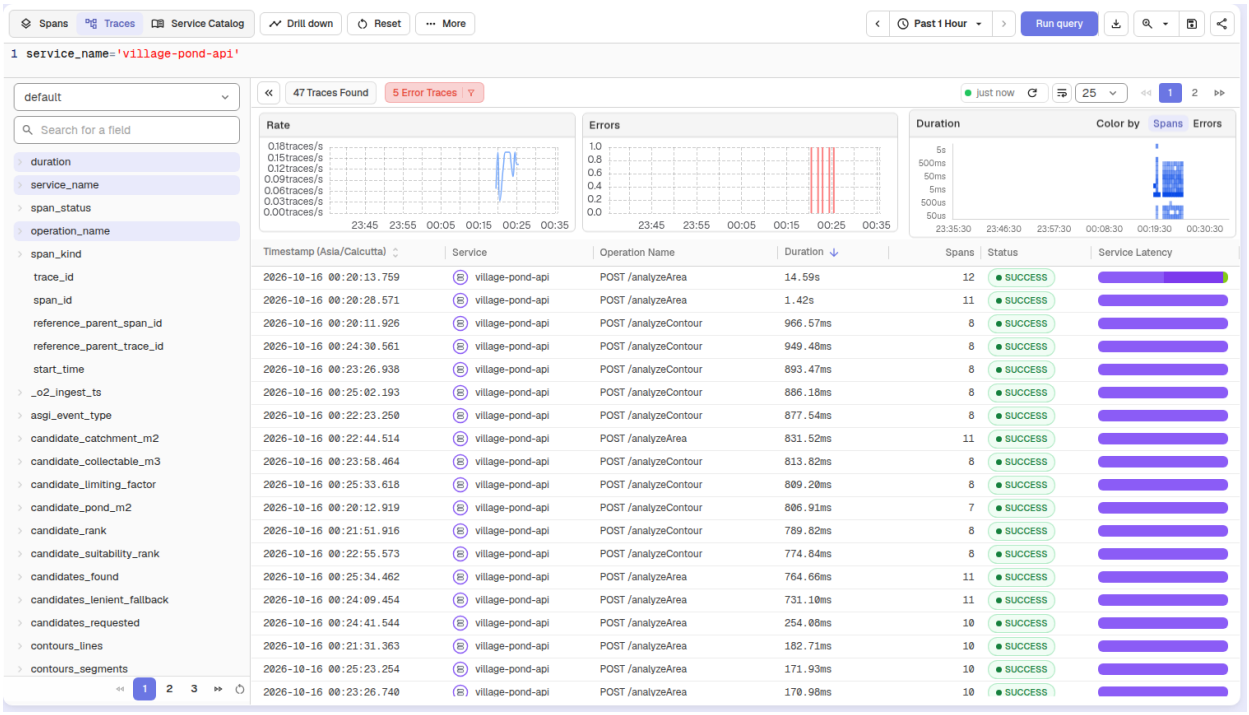


Figure 2: Traces view in OpenObserve, filtered with `service_name='village-pond-api'` and sorted by duration. The panels at the top show request rate, errors and a duration heat map over the last hour; below, each row is one request with its span count. 47 traces were recorded. The durations separate the request types at a glance: the one area analysis that had to download a fresh elevation model (14.59 s, 12 spans), area analyses that also rendered a PNG map (0.73–1.42 s, 11 spans), contour-file uploads (0.77–0.97 s, 8 spans) and cached repeats of the same area (0.17–0.25 s, 10 spans). The five "error traces" are the deliberately invalid requests, whose `pond.dem_fetch` span recorded the bounding-box exception.

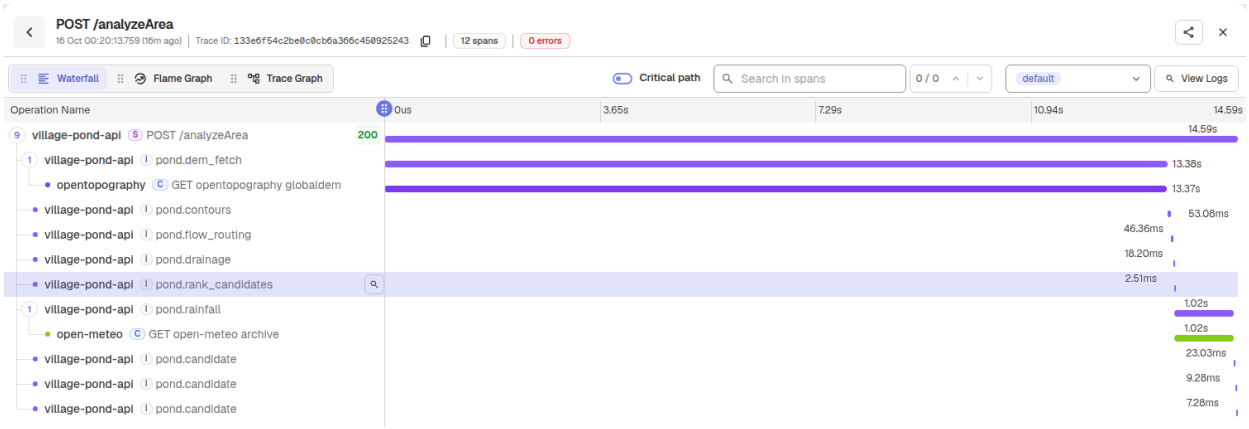


Figure 3: Waterfall of the slowest trace, the uncached `POST /analyzeArea` (trace `133e6f54...`). The tree shows the server span, the nine pipeline-stage spans as its children, and the two *client* spans for the external services, labelled with their peer service (`opentopography`, `open-meteo`). `pond.dem_fetch` and its child `GET opentopography globaldem` fill almost the whole bar (13.38 s of 14.59 s); the rainfall lookup takes 1.02 s, and contours, flow routing, drainage, ranking and the three candidate spans are thin slivers totalling about 0.16 s.

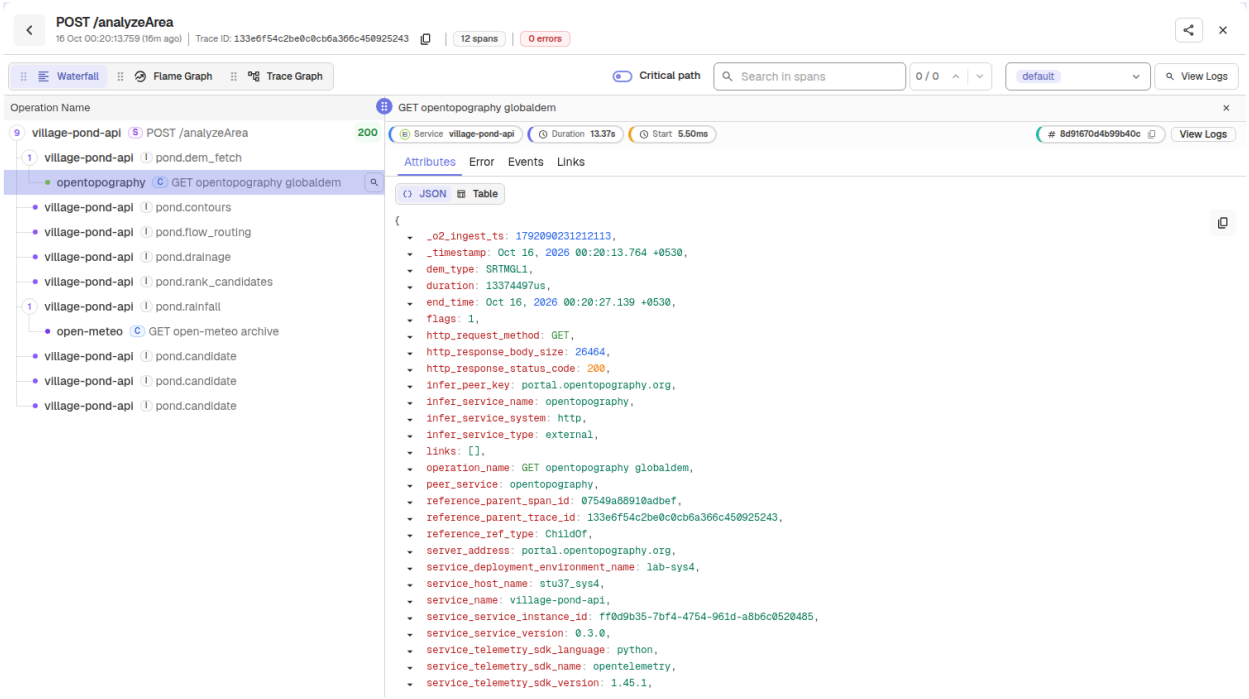


Figure 4: Attributes of the `GET opentopography globaldem` client span from the same trace. It records the HTTP method, the server (`portal.opentopography.org`), the DEM type (`SRTMGL1`), the response status (200) and body size (26 464 bytes) and the duration (13.37 s). Resource attributes added by the SDK identify where the span came from: `service_name=village-pond-api`, version 0.3.0, `deployment_environment_name=lab-sys4` and host `stu37_sys4`. There is no URL query string in the span, so the API key never leaves the server.

Logs

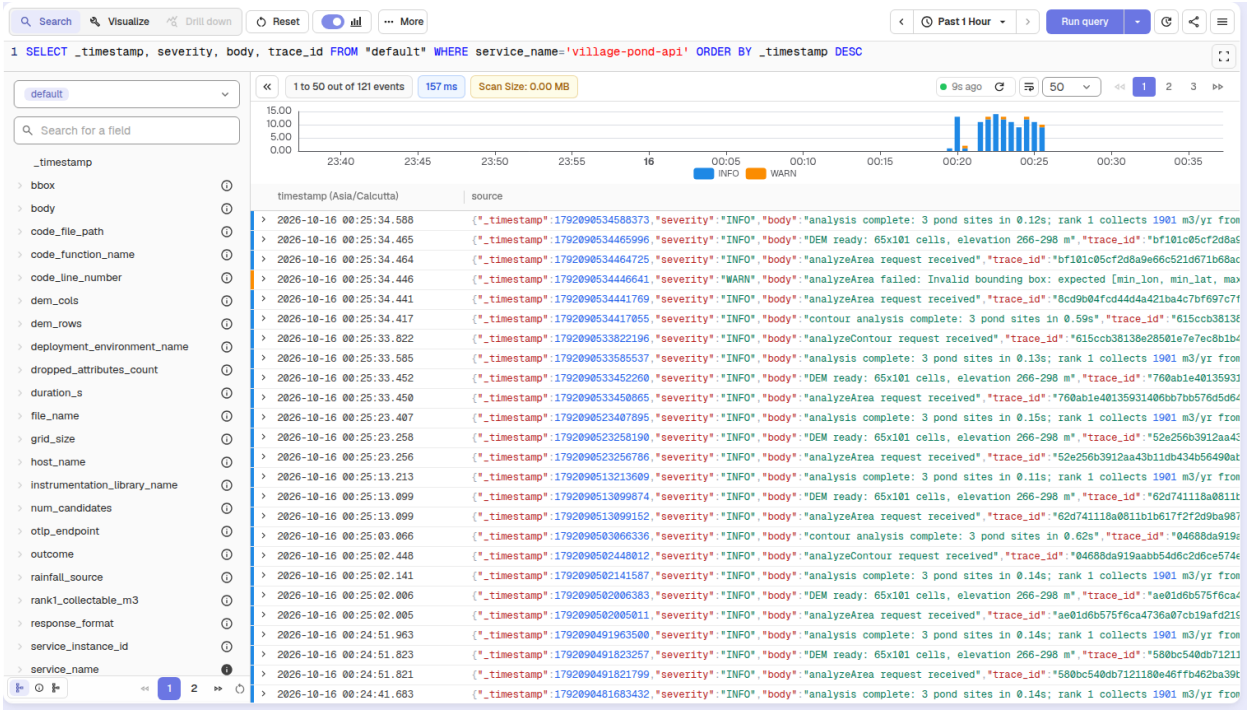


Figure 5: Logs view, queried with SQL over the `default` log stream. The bar chart shows log volume per minute split by severity (INFO in blue, WARN in orange); 121 records were stored. Every analysis writes the same three lines — *request received*, *DEM ready* (grid size and elevation range) and *analysis complete* (number of sites, time taken and the rank-1 collectable volume) — and each carries the `trace_id` of its request. The orange row is a rejected request with an invalid bounding box.

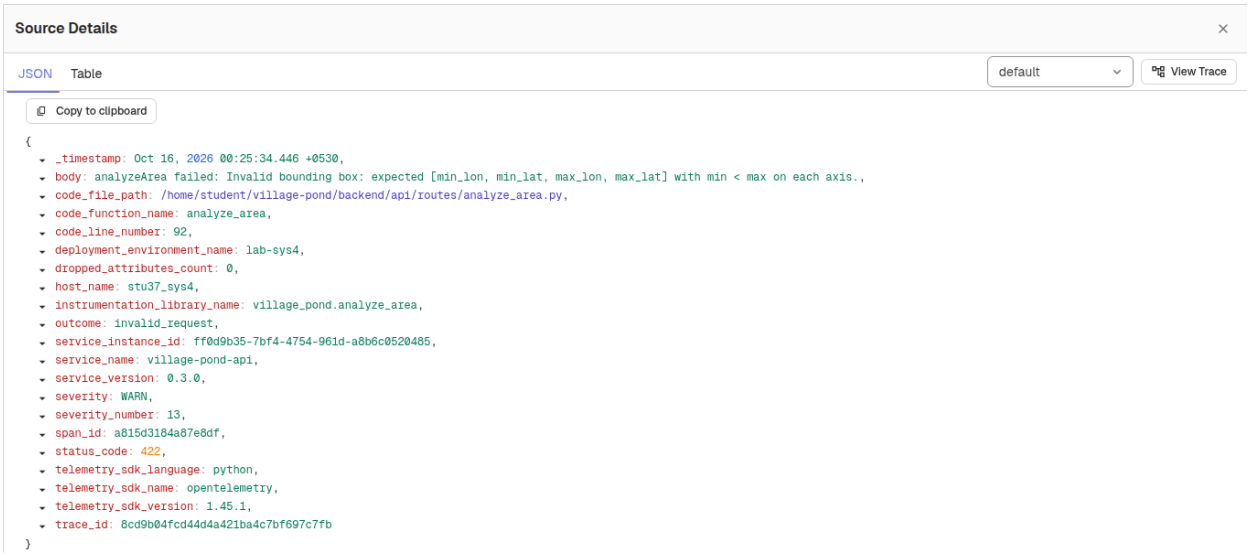


Figure 6: The WARN record from Figure 5 expanded. Besides the message it holds the structured attributes I attached (`outcome=invalid_request`, `status_code=422`), the source location added by the logging bridge (`analyze_area.py`, line 92), the resource attributes, and the `trace_id`/`span_id` of the request. The *View Trace* button uses that ID to open the trace the log line belongs to.

Metrics

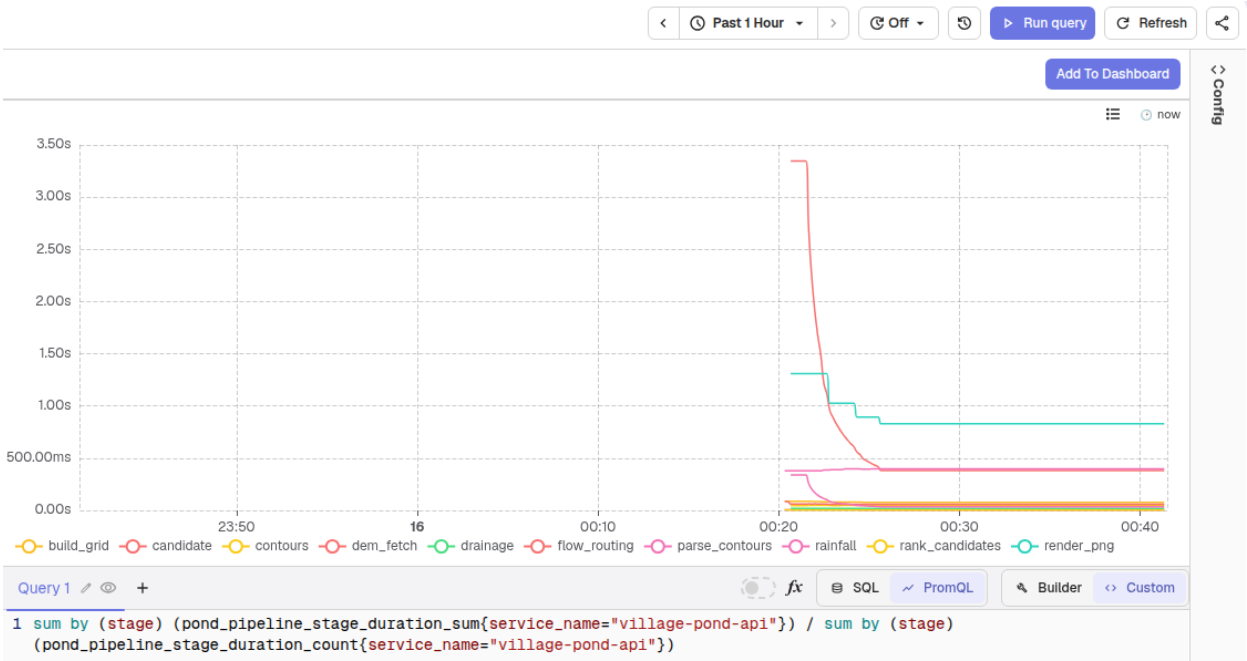


Figure 7: Average duration of each pipeline stage, computed in PromQL as `sum by (stage)` of the histogram's `_sum` divided by its `_count`. Because the histogram is cumulative, each line is a running average since the server started. `dem_fetch` (red) starts at about 3.3 s, when the one 13.4 s download is averaged with only a few cached requests, and falls to 0.38 s as cache hits accumulate. `render_png` (teal, about 0.83 s) ends up as the slowest stage on average, followed by `parse_contours` (about 0.4 s). All the terrain computation stages stay below 0.08 s.

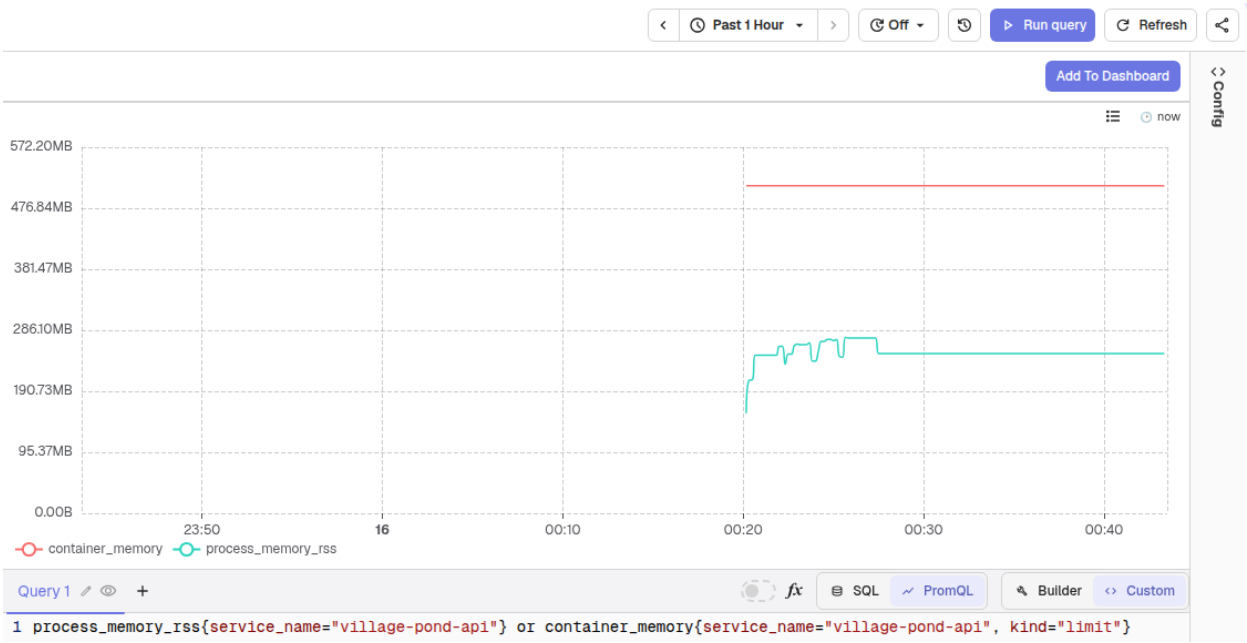


Figure 8: Resident memory of the API process (`process_memory_rss`, teal) against the container's hard limit (`container_memory{kind="limit"}`, red, 512 MiB), both from the custom observable gauges. RSS starts at about 150 MiB after a restart and steps up to 210–250 MiB as requests arrive, with the largest jumps coming from PNG renders. It then stays flat, which shows that the in-memory caches are not leaking.

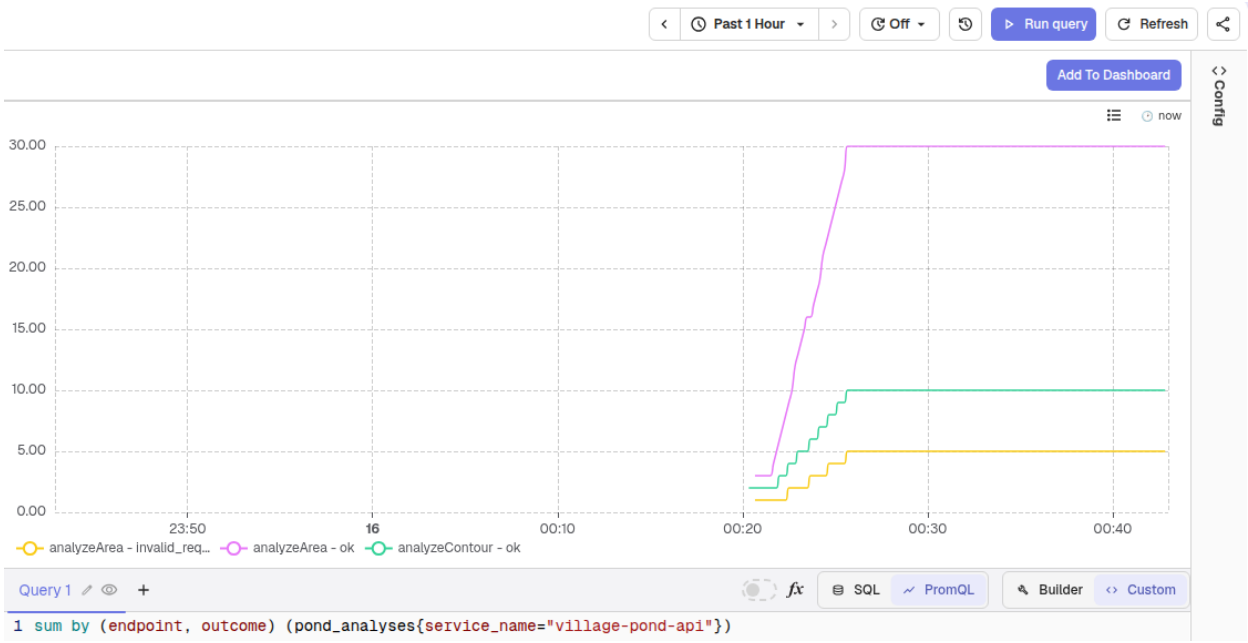


Figure 9: The `pond.analyses` counter grouped by endpoint and outcome (`sum by (endpoint, outcome)`). Each step is a request. By the end of the test run there were 30 successful area analyses, 10 successful contour analyses and 5 area requests rejected as `invalid_request`. There were no `upstream_error` outcomes.

From the web page to the trace

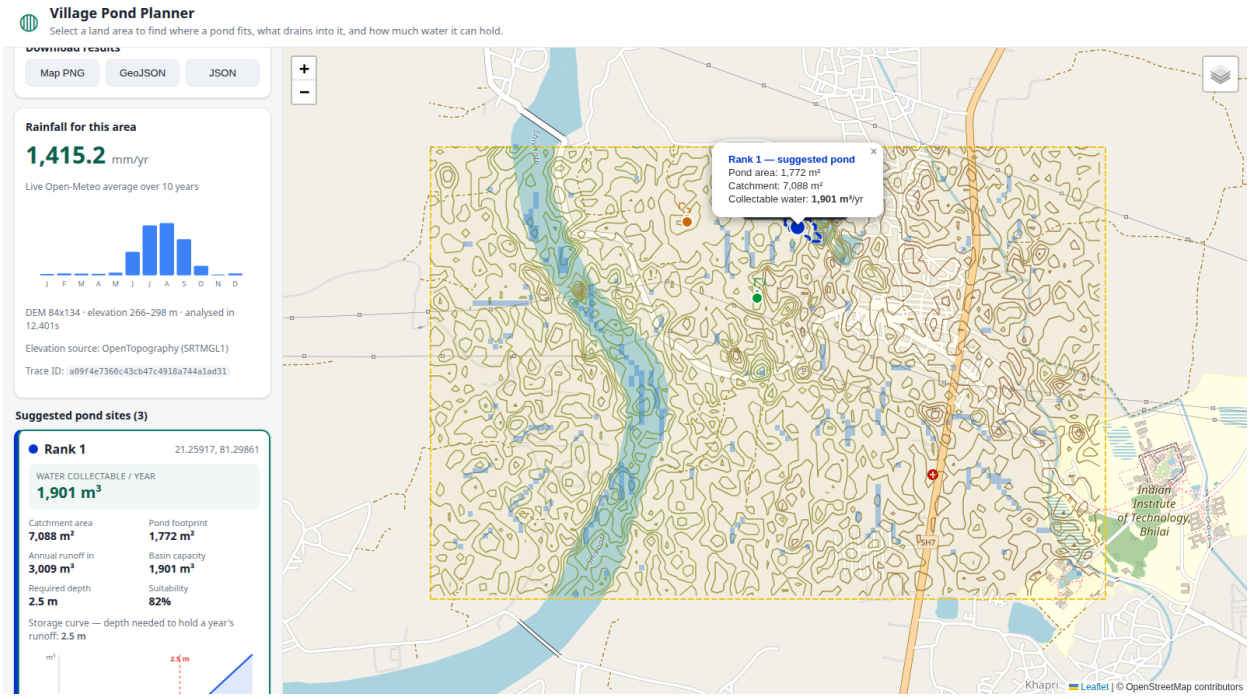


Figure 10: The application on sys4 after analysing the sample area from the browser (84×134 elevation grid, 12.4 s, three suggested sites). The results panel shows the request's trace ID, `a09f4e7360c43cb47c4918a744a1ad31`, under the DEM and rainfall summary.

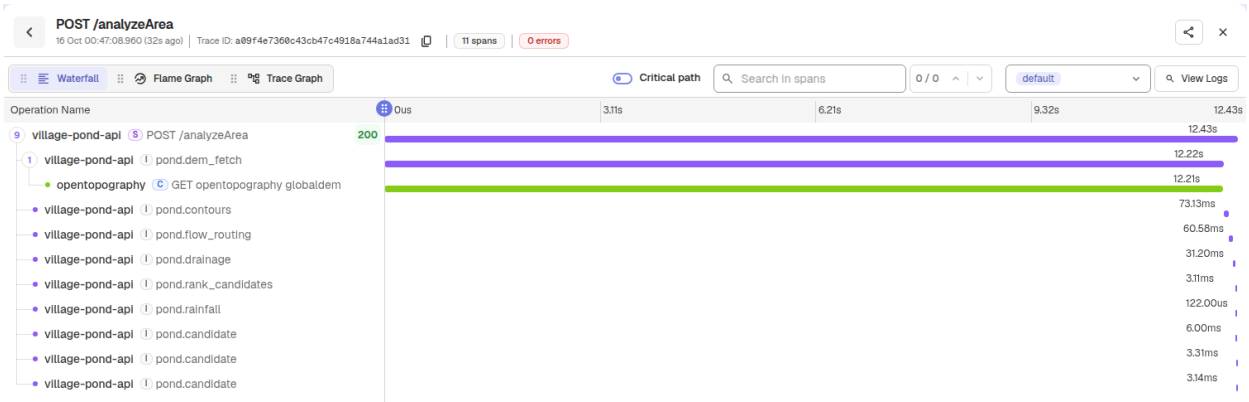


Figure 11: The same trace ID opened in OpenObserve. The request took 12.43 s, and 12.21 s of that was the OpenTopography download. The rainfall span took only 122 μs this time, because the rainfall for this location was already in the cache. The trace has 11 spans rather than 12, because a cached rainfall lookup makes no outgoing call and so has no client span.

6 What the data told me

Where the time actually goes. Table 3 is taken from the trace of the first analysis of the sample area on sys4. Before instrumenting I would have guessed that flow routing or the pond search was the expensive part. The trace says otherwise: **92% of the request is waiting for OpenTopography**, the rainfall archive adds another second, and all of my own computation together takes roughly 0.15 s. Repeating the same request returned in about 0.16 s, because the

elevation model was served from the in-memory cache (the `dem.cache_hit` attribute confirms it, and over the whole test run the `pond.dem.requests` counter recorded 29 cache hits against a single miss). Figure 7 shows the same thing: the running average of `dem_fetch` falls from 3.3 s to under 0.4 s as cached requests arrive. That figure also taught me something about reading metrics. An average over the whole run hides the one slow request, so the trace is what shows how slow an uncached request really is. So the caching I added in the previous assignment was the right optimisation, and the place to spend any further effort is the external download, not the algorithms.

Span	Duration (ms)
`POST /analyzeArea` (whole request)	14 590
`pond.dem_fetch` → `GET opentopography globaldem`	13 381 (13 375)
`pond.rainfall` → `GET open-meteo archive`	1 017 (1 016)
`pond.contours`	53
`pond.flow_routing`	46
`pond.drainage`	18
`pond.rank_candidates`	3
`pond.candidate` ×3	23, 9, 7

Table 3: Span durations for one uncached area analysis on sys4.

Memory headroom. The memory gauges (Figure 8) showed the API starting at about 150 MiB and rising to about 250 MiB once a PNG had been rendered, because matplotlib is only loaded on the first image request. That is a concrete justification for loading it lazily. It also shows how close an always-loaded plotting library would bring the service to the 512 MiB limit, which it shares with another lab's Node.js server on sys4. The container gauge's `used` series read about 440 MiB at the end of the run. That figure counts the other process and reclaimable file cache as well, but it still shows that the box has little room to spare.

Failures are visible instead of silent. Some failures in this app are handled gracefully on purpose, which normally makes them invisible. When the rainfall service fails, the app quietly uses a default value; I mark that span as an error and log a warning, so a fallback now shows up in the traces and in `pond.rainfall.lookups`. When OpenTopography's free quota (50 requests per day) runs out, the `pond.upstream.errors` counter records it with `kind=rate_limit`. Invalid requests are counted by outcome and logged as warnings with the reason, so a burst of bad input is easy to tell apart from a server fault.

Debugging a single request. The most useful workflow turned out to be the simplest: take the trace ID shown on the web page (or in the `X-Trace-Id` header), search for it in OpenObserve, and get both the full span waterfall and every log line from that request (Figures 10 and 11). Going the other way, the *View Trace* button on a log record (Figure 6) jumps from a warning to the request that caused it. Metrics answer "is something wrong and how often?"; the trace for one request answers "where exactly and why?"; and the logs carry the human-readable detail. Having all three tied together by the trace ID is what makes OpenTelemetry more than a logging library.

7 Problems I ran into

- **Telemetry lost on restart.** My first version flushed the exporters in an `atexit` hook, but nothing arrived when the server was stopped. Recent versions of uvicorn re-raise `SIGTERM` after shutting down, which kills the process before `atexit` runs. Moving the flush into FastAPI's

`lifespan` shutdown fixed it; I verified that the final batches of traces, metrics and logs are now sent when the app stops.

- **Noisy traces.** A 6 MB contour upload produced dozens of tiny ASGI `http receive` spans that buried the pipeline stages. Excluding the `receive`/`send` sub-spans in the FastAPI instrumentation left a clean waterfall of seven meaningful spans.
- **Secrets in telemetry.** Auto-instrumenting the HTTP client would have put the OpenTopography API key into span attributes and logs, so I wrote the client spans by hand and silenced the `httpx` logger.
- **Running a backend in 512 MB.** OpenObserve sized its worker pools from the 120 CPUs the container can see, and it refused to start until the admin password met its complexity rules. Small explicit limits fixed the first, a stronger password the second.

8 Conclusion

The Village Pond API now produces traces, metrics and logs through OpenTelemetry, exported over OTLP from the lab server to an OpenObserve backend on a second lab server, without slowing requests down (export happens in background batches). The data was immediately useful: it showed that nearly all of the response time is the external elevation download rather than my own code, confirmed that caching removes it, quantified the memory cost of rendering images against the 512 MB limit, and made the app's graceful fallbacks visible. Because the instrumentation only depends on the OpenTelemetry API and standard OTLP, it stays useful whichever backend is used next.

AI tool usage. I used Claude while working on this lab, mainly to help set up the OpenTelemetry SDK and the OpenObserve deployment and to track down the shutdown-flush problem. I reviewed and tested everything and can explain the implementation and the results myself.